DIGITAL DOCUMENT VERIFICATION SYSTEM (DDVS)

Comprehensive Software Design Analysis Report

Course: B.Tech Computer Science and Engineering	**Experiment Number:** Practical Lab 09
Subject: Software Engineering Laboratory	**Academic Year:** 2026 – 2027
Author / Student: Divya Kumari (202401100200133)	**Institution:** KIET Group of Institutions
Document Version: 1.0 (Final Academic Baseline)	**Date of Evaluation:** November 2026

Executive Summary

This technical report delivers an exhaustive architectural and qualitative design analysis of the Digital Document Verification System (DDVS), grounded in the formal IEEE Software Requirements Specification (SRS v1.0). The analysis evaluates the structural, modular, and dynamic mechanisms established across Practical Labs 01 through 08. Core Software Engineering principles—including three-tier layered separation of concerns, high functional cohesion, minimal stamp/data coupling, maintainability, architectural scalability, cryptographic reliability (SHA-256 integrity), and multi-layered access security—are rigorously examined. The report establishes theoretical validations, identifies current academic prototype constraints, and formulates a strategic engineering roadmap for production deployment.

1. Introduction and Scope of Design Analysis

1.1 Project Background and Problem Statement

In modern academia, corporate hiring, and governmental administration, digital credentials (certificates, diplomas, mark sheets, and identity credentials) are ubiquitously exchanged as PDF documents. However, digital files are inherently vulnerable to unauthorized alterations, desktop forgery, and credential tampering using standard editing software. Recipients lack an automated, rapid, and mathematically tamper-evident mechanism to authenticate digital credentials directly against the issuing authority without protracted manual correspondence.

1.2 Project Objectives

The Digital Document Verification System (DDVS) addresses this trust deficit by providing a web-based platform where authorized issuing organizations register electronic documents along with cryptographic integrity hashes, allowing employers, universities, and general verifiers to instantaneously evaluate authenticity via alphanumeric Verification IDs, scannable QR codes, or electronic PDF document uploads.

1.3 Scope of the Design Analysis

This analysis evaluates the logical architecture developed from the SRS requirements baseline. It investigates the structural cohesion of component boundaries, analyzes inter-module coupling, evaluates system survivability and fault-tolerance, and assesses compliance with established software quality metrics (ISO/IEC 25010 SQuaRE). In strict adherence to academic standards, all evaluations reflect the logical design without asserting unverified third-party deployment technologies.

2. Architectural Analysis and Structural Decomposition

DDVS is architected according to a standard **Three-Tier Logical Architecture**, strictly separating user interaction, transactional business logic, and persistent record storage. This tiering guarantees architectural modularity and eliminates tight coupling between client interfaces and database storage engines:

- **Tier 1: Presentation Tier (Web Client):** Comprises the user-facing web interface accessed through standard browsers. It encapsulates input capture (ID entry, camera QR scanning, and PDF uploading), client-side syntactic validation, and dynamic rendering of verification results (Verified, Tampered, Invalid, Not Found). It maintains no direct database connectivity and contains zero business evaluation logic.
- **Tier 2: Application Business Logic Tier (Application Server):** Acts as the central transaction controller. It encapsulates user authentication, session state management, role-based authorization, document issuance coordination, cryptographic SHA-256 hash generation, and verification outcome synthesis. It communicates with Tier 1 via secure HTTPS (TLS 1.3) RESTful protocols.
- **Tier 3: Data Persistence Tier (Relational Storage):** Encapsulates persistent state within three logical stores: User Store (D1), Document Store (D2), and Verification Log Store (D3). It enforces relational referential integrity, transactional ACID semantics, and append-only audit persistence.

3. Modularity and Separation of Concerns

Modularity measures the degree to which a software system is composed of discrete, self-contained units such that each unit encapsulates a single distinct aspect of functionality. In DDVS, modularity is manifested across seven logical components:

Component / Module	Primary Encapsulated Responsibility	Modularity Benefit
User Interface	User presentation, form capture, DOM updates	Presentation changes do not affect backend services
Account Management	Authentication, password hashing, RBAC	Centralized identity logic; easily swappable
Document Registration	Metadata ingestion, ID/QR generation	Issuance workflow isolated from verification
Document Verification	Query routing, status check, verdict synthesis	Core business rules isolated and independently testable
Integrity Hash Engine	Stateless cryptographic SHA-256 calculation	Pure utility; zero state, reusable across modules
Verification Log Manager	Append-only audit writes, compliance queries	Audit logging decoupled from transaction dispatch

Data Persistence	DAO interfaces, SQL queries, connection pooling	Database engine independence (Dependency Inversion)

4. Cohesion Analysis across System Modules

Cohesion characterizes the intra-module strength of functional relatedness among elements within a single component. Software Engineering taxonomy identifies seven levels of cohesion, ranging from lowest (worst) to highest (best): Coincidental, Logical, Temporal, Procedural, Communicational, Sequential, and Functional. The DDVS architecture is intentionally engineered to achieve **Functional Cohesion** across all operational modules:

- **Account Management (Functional Cohesion):** Every method inside this module (`register`, `login`, `changePassword`, `approveOrg`) contributes strictly to the single functional goal of managing authenticated identity and authorization.
- **Document Registration (Sequential & Functional Cohesion):** Operations follow a strict sequential pipeline: validate metadata ■■■ compute SHA-256 hash ■■■ generate verification ID ■■■ persist document. Each step consumes the output of the preceding step.
- **Integrity Hash Processing (Pure Functional Cohesion):** Performs exactly one mathematical operation: taking a binary byte stream and outputting an immutable 256-bit cryptographic digest. It maintains zero side-effects and zero external dependencies.
- **Verification Log Management (Functional Cohesion):** Dedicated strictly to audit record creation and history retrieval, ensuring audit compliance logic is never entangled with core verification branching.

5. Coupling Analysis and Interface Decoupling

Coupling measures the degree of interdependence between software modules. The classic coupling hierarchy ranges from highest/worst to lowest/best: Content Coupling, Common Coupling, Control Coupling, Stamp Coupling, and Data Coupling. In DDVS, low coupling is achieved through strict interface abstraction:

- **Elimination of Content & Common Coupling:** No component has direct access to the internal memory, instance variables, or global variables of another component. Global shared memory is entirely prohibited.
- **Encapsulated Data Persistence (Loose Data Coupling):** Business logic components never write raw SQL queries or depend on database table layouts. Instead, they interact via the `IDataAccess` interface, passing simple data transfer objects (DTOs) or primitive identifiers. This restricts coupling to minimal *Data Coupling* and clean *Stamp Coupling*.
- **Stateless Cryptographic Integration:** The `Document Verification` component invokes `IHashService.computeSHA256()` by passing only a stream handle and receiving a string hash. The hash engine possesses zero knowledge of who called it or why.

6. Evaluation of Non-Functional Software Quality Attributes

6.1 Maintainability and Extensibility

Maintainability is enhanced by strict adherence to the Single Responsibility Principle (SRP) and Dependency Inversion. If the cryptographic algorithm needs to be updated (e.g., migrating from SHA-256 to SHA-512 or post-quantum algorithms), only the `Integrity Hash Processing` component requires modification. The verification coordinator and database stores remain untouched.

6.2 Architectural Scalability

The logical architecture inherently supports horizontal scalability. Because the Application Server is designed to be stateless (session authentication tokens are validated per-request or via distributed token caches), multiple application server instances can operate behind a network load balancer. Document verification queries are read-heavy transactions; relational read-replicas and B-tree indexing on `documentId` enable sub-second response times even under heavy verification traffic.

6.3 Reliability and Fault Tolerance

System reliability is guaranteed through deterministic verification algorithms (Lab 07) and guaranteed audit logging. The system handles erroneous, tampered, or revoked inputs gracefully without crashing, returning formal standardized statuses. Transactional ACID semantics ensure that document registration and verification history logging are atomic operations.

6.4 Multi-Tier Security Architecture

Security considerations are embedded across all three tiers: (1) HTTPS / TLS 1.3 encryption across public networks; (2) One-way cryptographic password hashing (bcrypt / Argon2) preventing credential exposure; (3) Role-Based Access Control (RBAC) restricting document registration to approved organizations; (4) Cryptographic SHA-256 hashing providing mathematical proof against byte-level document tampering; and (5) Append-only verification history logging ensuring full forensic traceability.

7. Design Limitations and Engineering Roadmap

A critical, intellectually honest analysis acknowledges the realistic constraints of the current academic design baseline:

Identified Design Limitation	Root Cause / Impact	Proposed Engineering Improvement
Centralized Trust Authority	DDVS relies on a central relational DB; if compromised, hashes could theoretically be modified by root DB admins.	Anchor verification hashes to a public or consortium Blockchain ledger for decentralized immutability.
File Storage Scalability	Storing large electronic files in database BLOBs creates severe I/O bottlenecks and storage exhaustion.	Offload file binaries to secure Object Storage (e.g., S3/GCS) and store only cryptographic URI references.
Absence of Automated OCR	Scanned paper images cannot be verified if the text is altered prior to scanning.	Integrate an Optical Character Recognition (OCR) and layout analysis pipeline for printed paper documents.
Public Guest Verification Privacy	Public verification (FR-30) allows anyone with a valid ID to view certificate recipient names.	Incorporate Zero-Knowledge Proofs (ZKP) or selective disclosure to confirm authenticity without revealing personal data.

8. Conclusion and Design Quality Verdict

The architectural evaluation demonstrates that the Digital Document Verification System satisfies core Software Engineering benchmarks of high design quality. The three-tier structure enforces modular encapsulation, components demonstrate high functional cohesion, and interface-driven dependencies minimize coupling. Cryptographic integrity checking via SHA-256 and guaranteed audit logging ensure dependable protection against credential fraud. While future enhancements like decentralized ledgers and OCR integration will expand operational reach, the formulated architecture provides a robust, scalable, and maintainable software foundation.